UC Berkeley
MEng IEOR FinTech Capstone

Final Capstone Report

Reinforcement Learning for Cryptocurrency Trading with Path Signatures

---

Team Name: SinFech Research
Team Number: 232I

Team Members

Chenyu Tang
Chuttarsing Emeryc
Jiayang Hong
Mahiou Dany

Contents

Executive Summary **1**

1 Section 1: Business and Contextual Analysis **1**
1.1 Industry and Stakeholder Context 1
1.2 Problem Value and Project Scope 2
1.3 Engineering Leadership Considerations 2
1.4 Risks, Ethics, and Deployment Constraints 2

2 Section 2: Technical Project Outcomes **3**
2.1 Introduction 3
2.2 Technical Background 3
2.2.1 Core Innovation & Motivation 3
2.2.2 Problem Statement 4
2.2.3 Related Work 4
2.3 Data & Feature Engineering 5
2.3.1 Data Pipeline 5
2.3.2 Engineering the Path Signatures 5
2.4 Methodology 6
2.4.1 Capital-Aware Trading Environments 6
2.4.2 Advanced Q-Learning (D3QN) 6
2.4.3 Continuous Baselines (PPO & SAC) 7
2.4.4 Walk-Forward Evaluation Protocol 7
2.5 Experiments 8
2.5.1 Counterfactual Volume-Timing Representation Experiment 8
2.5.2 Representation and Benchmark Results 10
2.5.3 Signature Coordinate Ablation 10
2.6 Limitations and Future Work 11

References **12**

Executive Summary

This capstone project studies whether reinforcement learning agents can trade cryptocurrency more robustly when their market observations include path-signature features rather than only raw price windows. Cryptocurrency markets are noisy, non-stationary, and sensitive to transaction costs, so a trading model that performs well in one historical period can fail quickly out of sample. Our work addresses this problem by combining a capital-aware trading environment with geometric time-series representations from Rough Path Theory.

The project implements a reproducible Bitcoin trading pipeline covering data preparation, feature engineering, environment design, agent training, and walk-forward evaluation. The environment tracks cash, position, equity, fees, slippage, and exposure limits, which makes the experiment closer to a constrained trading system than a simple buy/hold/sell classifier. On the modeling side, we compare an upgraded D3QN agent using prioritized replay and n-step returns against PPO and SAC baselines. Path-signature observations summarize recent market windows through low-dimensional features that encode trend, volatility, and lead-lag structure.

The results support the value of the representation and the evaluation framework, but they do not justify deployment as a live trading strategy. Signature features show smaller measured generalization gaps than raw or LSTM-processed windows in the reported representation comparison, and the improved D3QN raises average out-of-sample performance relative to its baseline. However, PPO remains stronger on worst-fold robustness, and all results depend on historical Bitcoin regimes, fixed execution assumptions, and simulated evaluation. The main contribution is therefore a controlled, reproducible foundation for future multi-asset portfolio research rather than a production trading system.

1 Section 1: Business and Contextual Analysis

1.1 Industry and Stakeholder Context

Algorithmic trading sits in a business context where technical performance, operational discipline, and risk communication are inseparable. A model that appears profitable in a backtest is not automatically valuable to a trading desk, research group, or portfolio manager. It must also be explainable enough to review, reproducible enough to audit, and conservative enough that its limitations are visible before capital is placed at risk. This is especially important in cryptocurrency markets, where liquidity, volatility, exchange structure, and investor behavior can change quickly.

The immediate stakeholders for this capstone are the project advisor, faculty reviewers, and technically trained readers who need to evaluate whether the methods are sound. A second stakeholder group is future quantitative researchers who might extend the work into a broader portfolio setting. For them, the value of the project is not a single headline return; it is the construction of a disciplined experimental foundation. The project also needs to be understandable after the original experiments are complete, because future work depends on knowing which decisions were deliberate and which were simplifying assumptions.

These stakeholder needs shape the report. We present the project as a research system for studying representation and reinforcement learning under trading constraints, not as an investment product. That distinction matters because simulated returns can be misread by non-specialist audiences. The business question is therefore not simply whether the agent can make money in historical Bitcoin data. The more useful question is whether the project creates a reliable basis for comparing state representations, agent architectures, and evaluation protocols before any live deployment is considered.

1.2 Problem Value and Project Scope

The project addresses a real problem in applied financial machine learning: models often perform well during development but degrade when market conditions change. In reinforcement learning, this problem is amplified because the agent learns from delayed rewards and can discover policies that exploit artifacts of the simulator rather than durable market structure. A trading environment that ignores transaction costs, exposure limits, and capital accounting can reward behavior that would not survive realistic execution.

This project tries to narrow the gap between simplified RL trading experiments and the way a strategy would actually be evaluated in finance. The environment accounts for cash, position, equity, fees, slippage, and exposure limits, so the agent cannot earn rewards by ignoring basic trading constraints. Signature features serve a different purpose: they compress recent market paths into trend, volatility, and lead-lag information instead of giving the agent only raw price windows. The result is a testbed for asking whether a representation holds up under more realistic trading assumptions.

Scope control was a central business and engineering decision. The long-term application is dynamic portfolio allocation across multiple cryptocurrency assets, but solving that full problem immediately would combine too many uncertainties: cross-asset correlations, liquidity differences, action-space scaling, risk budgeting, and portfolio-level constraints. We therefore chose a single-asset Bitcoin setting. Bitcoin is liquid, widely observed, and available across a long historical sample, making it a reasonable first environment for isolating the interaction between representation, reward design, and agent behavior. This reduction keeps the report focused on whether the learning stack is robust before expanding to portfolio allocation.

1.3 Engineering Leadership Considerations

The main engineering challenge was keeping the project tied to evidence instead of the most attractive backtest. In financial machine learning, different seeds, folds, features, and hyperparameters can produce very different results, which makes cherry-picking easy. We addressed this early by fixing the evaluation protocol before using it to compare models. Walk-forward folds, multiple seeds, and a worst-fold Sharpe criterion make the comparison less dependent on one favorable market period.

The team also had to balance ambition with delivery risk. Reinforcement learning, path signatures, and cryptocurrency trading are each technically complex. Combining them without a clear architecture would have made debugging and interpretation difficult. We therefore separated the system into data preparation, path construction, signature transformation, environment accounting, agent training, and evaluation. This modular design allowed experiments to change without rewriting the entire stack, and it made failures easier to localize as representation, reward, agent, or fold-specific market-regime issues.

Communication was another leadership concern. The report must be readable by a mixed technical audience: reviewers may understand machine learning, finance, or software systems at different levels of depth. For that reason, the results are framed around robustness, generalization, and constraints rather than only around return. The project also avoids presenting the improved D3QN as the final best model when the evidence is more nuanced. PPO remains stronger on worst-fold robustness, while the upgraded D3QN improves average performance relative to its baseline. Stating that distinction clearly prevents a promising engineering result from being overstated as a deployment-ready conclusion.

1.4 Risks, Ethics, and Deployment Constraints

The most important risk is backtest overfitting. Even with walk-forward evaluation, historical testing cannot guarantee future performance. Cryptocurrency markets have experienced exchange

failures, regulatory changes, liquidity shocks, leverage cycles, and regime shifts that may not repeat in the same form. A strategy trained on past Bitcoin behavior can also learn relationships that are accidental rather than structural. The results should therefore be interpreted as experimental evidence about a research pipeline, not as proof of a stable trading edge.

A second risk is model risk. Deep reinforcement learning agents are difficult to inspect, and their behavior can change sharply outside the training distribution. Path signatures may give the agent a better-structured observation space, but they do not make the learned policy safe by themselves. Any deployment-oriented version of this work would need stricter cost and liquidity assumptions, drawdown limits, stress tests, paper trading, monitoring, and human override procedures. It would also need clear rules for when a model should be retrained, paused, or retired.

There are also ethical communication constraints. Simulated returns can influence readers even when no financial advice is intended. The report therefore needs to state assumptions clearly and avoid implying that an academic backtest is a recommendation to trade. Responsible presentation includes showing negative folds, uncertainty across seeds, and limitations of the single-asset setup. This is not just a legal concern; it is part of engineering integrity. A model that is useful for research can still be inappropriate for unsupervised financial decision-making.

Finally, deployment would require much more than model accuracy. A live system would need reliable data feeds, exchange connectivity, execution controls, secure key management, compliance review, logging, and incident response. Those systems are outside the scope of this capstone. The next step is to extend the research framework to more assets, more realistic execution assumptions, portfolio-level risk controls, and validation across changing market regimes.

2 Section 2: Technical Project Outcomes

2.1 Introduction

This section documents the technical outcomes of the project: the feature representation, trading environment, agent implementations, and evaluation results. The underlying problem is sequential decision-making under delayed rewards, changing market regimes, transaction costs, and downside risk. This makes Reinforcement Learning (RL) a suitable framework, while also exposing a central weakness of Deep Reinforcement Learning (DRL) in finance: agents can overfit noisy price histories and fail out of sample.

The system addresses this weakness from both the environment and representation sides. We compare an upgraded D3QN agent with PPO/SAC baselines inside a "capital-aware" Gymnasium environment that models transaction fees, slippage, exposure, and equity dynamics. We also integrate **Path Signatures**—derived from Rough Path Theory—as geometric time-series features intended to separate structural price dynamics from market noise. Using Bitcoin data from 2010–2024, we test whether signature-based observations improve generalization and risk-adjusted performance relative to raw market features.

2.2 Technical Background

The background for this study sits at the intersection of two questions: how DRL agents can learn trading policies from noisy cryptocurrency data, and whether path signature features can provide a more stable representation of recent market dynamics.

2.2.1 Core Innovation & Motivation

This project combines DRL with path signature features. DRL provides the autonomous decision-making engine for maximizing long-term rewards, while Path Signatures provide a mathematical

representation of time series rooted in Rough Path Theory [14]. The signature of a path is an infinite sequence of iterated integrals that encodes the path's geometric structure and can approximate continuous functionals of the path through linear models.

Recent applied work [19] motivates signatures as structured features for noisy, regime-shifting time series.

This real-world forecasting application suggests that path signatures can be useful when observations are noisy and regimes shift. We therefore study whether signature features can give a DRL trading agent a more structured observation space by combining geometric summaries of recent price paths with RL-based policy optimization.

2.2.2 Problem Statement

Financial time series, especially cryptocurrencies, are noisy and non-stationary. Standard RL agents fed with raw price windows can struggle to extract stable patterns, often fitting training-period noise (In-Sample overfitting) and generalizing poorly to unseen data (Out-Of-Sample degradation).

We isolate this challenge in a controlled Bitcoin setting before extending the framework to multi-asset allocation.

The Signature Solution: Path Signatures have been used in machine learning to represent time series through their geometric structure, including trends, volatility, and lead-lag dynamics. Two theoretical properties motivate their use here: linear functionals of sufficiently rich signatures can approximate broad classes of continuous path functionals [12, 16], and the full signature characterizes bounded-variation paths up to tree-like equivalence [15]. In practice, reparametrization invariance reduces sensitivity to timing changes that do not alter the path's geometry, while time augmentation can be added when clock time itself is informative. We hypothesize that pairing these features with DRL can improve out-of-sample generalization.

2.2.3 Related Work

Our approach lies at the intersection of two distinct research domains: the application of Reinforcement Learning to quantitative finance and the use of mathematical topology (Path Signatures) for machine learning.

Reinforcement Learning in Finance: The use of RL for algorithmic trading has been the subject of numerous recent publications, often employing architectures based on continuous algorithms like PPO [1] or discrete ones like DQN [2]. Recent surveys of DRL/RL in trading and quantitative finance emphasize persistent methodological issues around state design, reward specification, transaction costs, and risk-adjusted evaluation [8, 9]. A broader systematic survey of deep learning in stock-market applications further stresses the importance of backtesting practice and domain-specific evaluation metrics [10]. A common limitation in simplified experimental setups is that the trading decision is reduced to directional or position labels (+1 to buy, 0 to hold, -1 to sell) without sufficiently modeling compounding, drawdown, bankruptcy risk, or market frictions. Our architecture diverges from these works by coupling our agents (PPO, SAC, and our upgraded D3QN with Prioritized Experience Replay) to a "capital-aware" environment that explicitly models transaction costs and imposes maximum exposure ratios, making risk and exposure constraints part of the learning problem.

Rough Path Theory and Financial Feature Extraction: The mathematical foundation of our state representation relies on the seminal work of Lyons [14] on differential equations driven by rough signals. Hambly and Lyons later established the uniqueness of signatures for bounded-variation paths up to tree-like equivalence [15], while later work develops the view of signatures as universal features for path-dependent learning and payoffs [12, 16]. Recent surveys and primers

further position signatures as practical, geometry-aware features for streamed machine-learning data [11, 12], while Morrill et al. formalize practical design choices for multivariate time-series signature feature extraction [13]. Gyurkó et al. [18] applied signatures to financial data streams and studied how they extract structured information from market microstructure noise.

Beyond financial data, the recent paper by Gu, Guo, Jacobs, Kaminsky, and Li [19] demonstrates that signature-based forecasting can be deployed in a large-scale logistics setting. Their work reports more than a fivefold improvement in prediction accuracy for Amazon transportation marketplace rates and more than $50 million in estimated annualized savings, illustrating the practical value of signatures for non-stationary operational time series.

Our Contribution: While current literature often uses signatures for standard classification tasks (e.g., handwriting recognition), this project integrates them into the observation space of DRL trading agents. Unlike studies relying solely on summary statistics or raw price windows, our architecture exposes the agent to second-order signature cross-terms, such as the Lévy area. These terms encode volatility and lead-lag correlations that may be obscured in raw price windows.

2.3 Data & Feature Engineering

2.3.1 Data Pipeline

Our data pipeline uses historical Bitcoin data spanning from 2010 to 2024. We avoid look-ahead bias by keeping every evaluation window temporally ordered. When a single fixed split is used for diagnostics, the In-Sample (training) period covers 2014 to 2020, while the Out-Of-Sample (testing) period spans 2021 to 2024. The final multi-seed model comparison uses the rolling walk-forward folds defined in Section 2.4.4.

Single-Asset Baseline Design: We deliberately scoped the final experiments to Bitcoin. This design isolates the interaction between the trading environment, DRL agents, and signature feature pipeline before introducing cross-asset portfolio effects. The pipeline uses OHLC-derived path coordinates and account-state variables, so we can evaluate whether geometric path features improve risk-adjusted trading performance under controlled market data and execution assumptions.

Feature Scope: The main trading baseline experiments focus on price-path coordinates that can be computed consistently across the full 2010–2024 sample: normalized log price, log returns, rolling volatility, and rolling mean return. This keeps the baseline evaluation reproducible and avoids mixing signals with shorter or less reliable histories. The representation experiment and coordinate ablation later test whether adding or replacing channels such as cumulative volume and high-low range improves the semantic content of the resulting signature features.

2.3.2 Engineering the Path Signatures

The Signature Transform: For a continuo us path $X : [0, T] \to \mathbb{R}^d$, the signature is defined as the collection of iterated integrals:

$$S(X)_{0,T} = \left(1,\ S^{(1)}(X),\ S^{(2)}(X),\ \ldots\right), \quad S^{(k)}(X)_{i_1,\ldots,i_k} = \int_{0<t_1<\cdots<t_k<T} dX_{t_1}^{i_1} \cdots dX_{t_k}^{i_k}$$

where $S^{(1)}$ encodes net displacement (trend), $S^{(2)}$ encodes lead-lag correlations and volatility patterns, and higher-order terms encode regime transitions and oscillation structure.

Implementation details: We compute the log-signature (a numerically stable variant via the Baker-Campbell-Hausdorff formula) using `pysiglib`, truncated at degree $N \in \{2, 3\}$. For base dimension d and degree N, the output dimension is $\sum_{k=1}^{N} d^k$. Degree-3 signatures

with time augmentation (appending t as an extra coordinate) are used in our best-performing configuration.

Architecture: Our custom `PathBuilder` transforms each 24-hour OHLC window into a path tensor using four channels: log-price normalization ($\log P_t - \log P_0$), log-returns ($\log P_t - \log P_{t-1}$), rolling volatility, and rolling mean return. The `SignatureObsWrapper` then applies the log-signature transform and appends account state features (cash ratio, position ratio, equity return, turnover ratio), producing the final observation vector $\mathbf{o}_t \in \mathbb{R}^{124}$.

Candidate Additional Channel — High-Low Range: For each bar t, the high-low range is defined as $h_t = \ln(H_t/L_t)$, where H_t and L_t denote the intra-period high and low prices respectively. Unlike `log_return`, which captures only net displacement, h_t measures intra-period oscillation. As a path coordinate, it activates cross-terms with `log_return` that distinguish clean directional moves from choppy, mean-reverting bars with identical net returns. This channel is evaluated in the ablation study in Section 2.5.2.

2.4 Methodology

To train our agents, we developed a modular architecture that separates market simulation from decision-making algorithms. This decoupling lets us benchmark discrete and continuous agents under identical market conditions.

2.4.1 Capital-Aware Trading Environments

The Flaw of Legacy Environments: Earlier simplified trading environments treated actions as buy/hold/sell labels without full capital accounting, which produced unrealistic in-sample returns in early tests.

Our Implementation: We built a custom, fully continuous `Gymnasium` environment with ledger-style accounting for cash, positions, equity, fees, and execution effects. Continuous-control agents output target exposure directly, while the discrete D3QN agent maps action bins onto the same exposure and accounting interface.

- **State Tracking:** The environment strictly tracks `cash`, `position` (fractional asset holdings), and total `equity` at each timestep t.
- **Real-World Mechanics:** We integrated transaction costs (modeling exchange maker/taker fees at 0.1%), slippage mechanisms to simulate execution inefficiency, and fractional actions.
- **Risk Management:** We introduced a `max_exposure_ratio` parameter. This prevents the agent from allocating 100% of its capital to a single trade and makes exposure control part of the learned policy, which is necessary for future portfolio allocation experiments.

2.4.2 Advanced Q-Learning (D3QN)

For our primary discrete-action agent, we engineered a Deep Double Dueling Q-Network (D3QN), combining the DQN value-function framework [2], Double DQN's decoupled action selection and evaluation [3], and the dueling value/advantage decomposition [4]. We discretized the action space into precise execution bins (e.g., aggressively short, mildly short, hold, mildly long, aggressively long) to map to our capital-aware ledger. Figure 1 summarizes the model architecture and its training feedback loop.

Why this architecture?

- **Dueling Network:** In trading, there are many market states where the optimal action is simply to "hold" and do nothing. The dueling architecture separates the estimation of the state-value $V(s)$ from the advantage of each action $A(s, a)$, allowing the agent to learn the inherent value of a market state independently of the action taken.

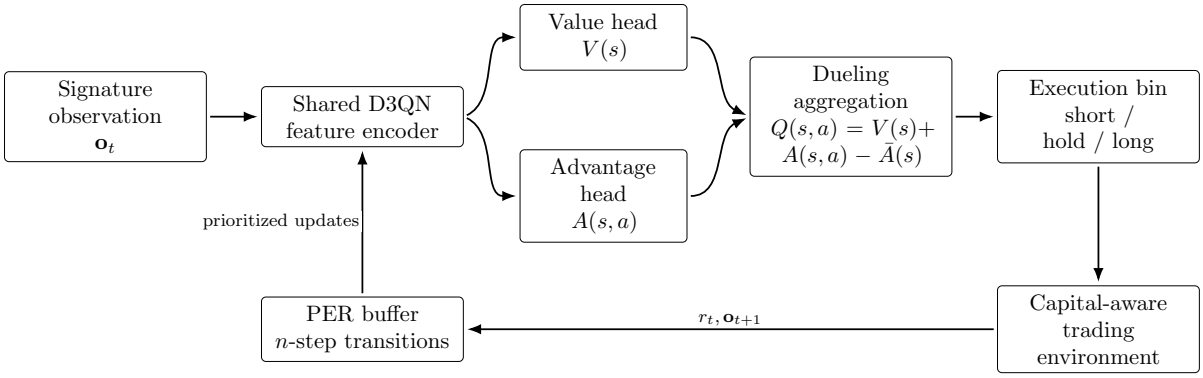


Figure 1: D3QN architecture used for the discrete trading agent. Signature-based observations are encoded by a shared network and split into dueling value and advantage streams, while prioritized replay and n-step returns feed informative transitions back into training.

- **Double DQN:** Financial rewards are highly volatile. Standard DQN overestimates Q-values due to the max operator. Double DQN mitigates this by decoupling action selection from action evaluation.

Recent Upgrades: To process the complex, geometric states generated by our `SignatureObsWrapper` efficiently, we upgraded the D3QN with **Prioritized Experience Replay (PER)** [5] and **n-step transition buffering** [7]. PER prioritizes transitions with larger learning signal, such as high TD-error observations that can arise around crashes or breakouts, instead of sampling all replayed experiences uniformly. Meanwhile, n-step returns help propagate the heavily delayed financial rewards back to the agent much faster than standard 1-step Q-learning.

2.4.3 Continuous Baselines (PPO & SAC)

While our optimized D3QN operates in a discrete action space, the longer-term portfolio direction requires dynamically assigning continuous weights to multiple assets. We therefore implemented two widely used continuous-control RL algorithms as benchmarks.

- **Proximal Policy Optimization (PPO):** Chosen for its stability and ease of tuning. PPO is an on-policy actor-critic algorithm that uses a clipped surrogate objective to prevent destructively large policy updates [1]. It directly outputs a continuous action $a_t \in [-1, 1]$ representing the target portfolio weight, making it our primary baseline.
- **Soft Actor-Critic (SAC):** Chosen for its sample efficiency and robustness. As an off-policy algorithm that maximizes both expected return and policy entropy, SAC encourages exploration [6]. This is useful in non-stationary crypto markets because it can reduce the risk of collapsing into a deterministic, suboptimal local minimum.

By comparing our signature-enhanced D3QN against these continuous baselines, we can separate the effect of signature-based feature engineering from the effect of the underlying RL optimizer.

2.4.4 Walk-Forward Evaluation Protocol

To reduce backtest overfitting and make the comparisons more reliable, we transitioned from basic single-run checks to a standardized evaluation framework.

Unified Walk-Forward Protocol: Financial regimes change dynamically. To reduce dependence on any single bullish or bearish phase, we use a rolling walk-forward design. The

dataset is divided into 3 distinct temporal folds ($f1$, $f2$, $f3$). For each algorithm, we execute training across 5 independent random seeds (42 through 46) per fold. For every (algorithm, fold, seed) triplet, the agent is trained exclusively on the fold's training window and subsequently evaluated on both the In-Sample (IS) and Out-Of-Sample (OOS) windows.

Standardized Evaluation Controls: To compare our discrete D3QN and continuous baselines (PPO, SAC) under the same protocol, all evaluations are deterministic. We enforce fixed indices using `fixed_windows=true` and `fixed_windows_seed=20240101`. During the evaluation phase, exploration is disabled ($\epsilon = 0.0$) to test the exploited policy across 50 episodes. Furthermore, training windows were run with `train_split=1.0` to use the available fold data without artificial shrinkage.

Ablation & Sweeps: To isolate the effect of our architectural upgrades, we conducted targeted ablation studies on the D3QN. We compared the baseline D3QN against variants equipped with Prioritized Experience Replay (PER) and n-step returns. We then swept risk parameters and action spaces, testing a 6-action discrete space alongside the selected memory hyperparameters (PER $\alpha = 0.6$, n-step=3) under the same walk-forward protocol.

Robust Selection Criterion: In quantitative finance, survival is more critical than maximizing average returns. Therefore, our model selection is no longer driven by the highest IS performance. Instead, before inspecting the fold outcomes, we define the primary selection score as

$$\min_{f \in \{f1, f2, f3\}} \overline{\text{Sharpe}}_{\text{OOS}}(f),$$

where the bar denotes the mean across the five random seeds within fold f. This makes the *worst-fold OOS Sharpe ratio* a pre-specified robustness metric rather than an ad hoc choice of any particular fold. We use Sharpe rather than raw return because it measures risk-adjusted performance and penalizes high-volatility strategies. Mean OOS Sharpe and mean OOS Return are used only as secondary tie-breaking metrics.

2.5 Experiments

2.5.1 Counterfactual Volume-Timing Representation Experiment

We use a controlled counterfactual volume-timing experiment to test whether the signature pipeline captures within-window temporal structure. The experiment is not intended as a trading backtest. Instead, it asks a narrower question on real Bitcoin OHLCV windows from our dataset: when the price path and total volume are held fixed, can the feature representation distinguish whether volume arrives earlier or later within the window?

Task design: We sample 24-bar Bitcoin windows from the same data source used in the main pipeline, with the training split running from January 1, 2014 to December 31, 2020 and the test split from January 1, 2021 to February 9, 2024. We draw 256 source windows from each split. For each source window, we keep `Close`, `High`, and `Low` unchanged, and we preserve total volume over the full window. We then generate two counterfactual classes: an early-volume class, in which more volume is concentrated in the first half of the window, and a late-volume class, in which more volume is concentrated in the second half. The perturbation is partial rather than absolute, so the classification problem is not trivial: the modified volume path preserves 55% of the original profile and blends in 45% of the target early- or late-volume pattern.

This setup isolates one representational question. Because price levels, highs, lows, and total window volume are held fixed, any successful classifier must use the timing of volume within the window rather than coarse endpoint statistics.

Feature comparison: We compare three input families using the same small MLP probe classifier. Keeping the downstream model fixed lets the comparison focus on the representation itself. The first family consists of nine summary features, namely final log return, mean log return, return volatility, maximum and minimum close, close range, and total, mean, and

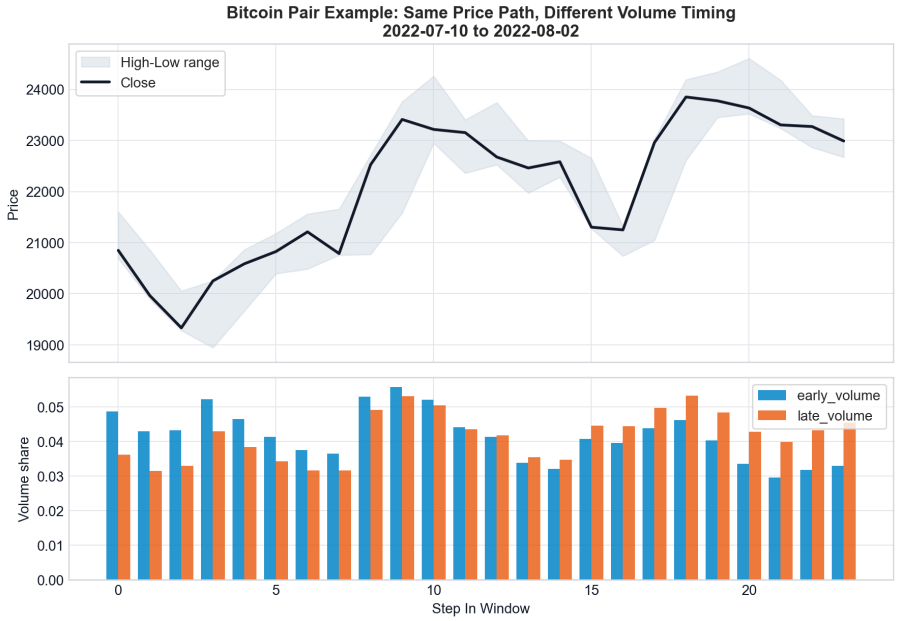


Figure 2: One counterfactual pair sampled from the Bitcoin dataset. The price path is unchanged, and only the within-window timing of volume differs.

standard deviation of volume. The second is the raw window, represented as a flattened 24-bar OHLCV tensor with 96 dimensions. The third is the signature representation from the repository stack built with `PathBuilder` and `LogSigTransformer`. Its channels are `log_price`, `log_return`, `normalized_cumulative_volume`, and `high_low_range`. Signatures are truncated at degree 2, time augmentation is enabled, lead-lag augmentation is disabled, and the resulting feature dimension is 15. All three feature families are passed to the same small MLP probe, so differences primarily reflect the input representation.

Feature Set	Dim	Train Accuracy	Test Accuracy
`summary`	9	0.5625	0.4961
`raw_window`	96	0.8652	0.6797
`signature`	15	0.9512	0.9141

Table 1: Counterfactual early-vs.-late volume-timing classification results. The signature representation achieves the highest accuracy while remaining low-dimensional.

Interpretation: The summary baseline is essentially at chance, which is expected because summary statistics discard the ordering of volume within the window. The raw flattened window retains some timing information and reaches about 0.68 test accuracy. The signature representation reaches about 0.91 test accuracy with only 15 dimensions. In this task, the signature stack retains the temporal structure that separates the two classes more effectively than either coarse aggregates or the raw flattened path.

This result is strong but not pathological. Earlier toy versions used a linear probe together with a fully rewritten early-vs.-late volume profile, which made the task unrealistically clean. The current version is deliberately harder: it uses a small MLP rather than a linear classifier, and the counterfactual volume path is only a partial perturbation of the original window. Under this more credible setup, the raw-window baseline captures some signal, but the signature representation still keeps a clear advantage.

Limitations and value: This experiment does not establish trading profitability, nor does it show on its own that D3QN policy learning will improve. The conclusion is narrower: the current

signature channels—especially `normalized_cumulative_volume`—retain information about when volume arrives within a bar sequence. That result supports the use of this state representation and complements the separate specialist report on downstream trading performance.

2.5.2 Representation and Benchmark Results

Signatures vs. Raw Features: In the reported feature comparison, signature-based representations have smaller measured generalization gaps than raw OHLC and LSTM-processed windows, as shown in Table 2.

Feature Type	OOS Sharpe	Gen. Gap
Raw price window (24×4 OHLC)	-0.61	1.39
LSTM-processed window	-0.48	1.12
Signature (degree 2)	-0.09	0.82
Signature (degree 3, full)	$+0.11$	0.71

Table 2: Generalization gap = IS Sharpe − OOS Sharpe. The signature configurations show smaller measured gaps in this comparison.

Rolling walk-forward, multi-seed evidence: Under a 3-fold × 5-seed protocol, PPO remains the most robust model (OOS Sharpe $= 0.4705 \pm 0.5948$, worst-fold Sharpe $= -0.1884$). Baseline D3QN yields OOS Sharpe $= 0.0945 \pm 0.8288$ (worst fold $= -0.9421$). The improved D3QN (6-action, PER $\alpha = 0.6$, n-step$= 3$) brings this to OOS Sharpe $= 0.3132 \pm 0.6321$, OOS Return $= 45.24\% \pm 54.62\%$, but worst-fold Sharpe remains negative ($= -0.4965$). As shown in Table 3, fold $f3$ is the minimum-Sharpe fold for all three reported configurations, so it determines the worst-fold metric in this experiment. D3QN improves average performance; PPO remains stronger on downside robustness.

Config	Fold	OOS Sharpe	OOS Return %
PPO baseline	f1	1.1079	66.35
	f2	0.4920	41.46
	f3	−0.1884	−9.28
D3QN (3-act, baseline)	f1	0.8650	68.60
	f2	0.3607	36.98
	f3	−0.9421	−26.35
D3QN (6-act, PER, n=3)	f1	0.8905	85.86
	f2	0.5456	67.45
	f3	−0.4965	−17.58

Table 3: Fold-wise OOS performance means (5 seeds per fold). The worst-fold score is the minimum OOS Sharpe across the three rows for each configuration; here it occurs on f3. Fold windows: f1 train 2014–2018 / test 2019–2022; f2 train 2015–2019 / test 2020–2023; f3 train 2016–2020 / test 2021–2024.

2.5.3 Signature Coordinate Ablation

The choice of path coordinates is structural: each coordinate determines which iterated integrals exist in the signature. Adding a channel activates new second-order cross-terms $S^{(2)}_{i,j}$; removing one eliminates them entirely. We tested three embedding modifications against the reference D3QN (6-action, PER $\alpha = 0.6$, n-step$= 3$) on fold `f1`, holding all other hyperparameters fixed (Table 4).

Config	5-seed Sharpe	5-seed Return %
Baseline	0.8905	85.86
D3_return_vol5 (drop `log_price`)	0.8728	95.38
F1_hlrange_rv10 (add `hlrange`, widen vol)	0.9072	84.72
MLP_D3_return_vol5 (MLP encoder)	0.8073	77.99
MLP_R5_volprof (swap to norm. volume)	0.8059	100.12

Table 4: Signature coordinate ablation on fold `f1`, 5-seed means.

- **D3_return_vol5** (drop `log_price`): `log_price` is non-stationary, so its level-1 signature term may dominate the representation and make cross-terms harder to interpret. Removing it makes the Lévy area $A^{\text{return,vol}}$ more prominent. On fold `f1`, this variant raises 5-seed return by +9.5pp relative to baseline, with a small decrease in Sharpe.
- **F1_hlrange_rv10** (add `hlrange`, widen vol window): $h_t = \ln(H_t/L_t)$ is a per-bar realized volatility estimator [17] that activates the cross-term $A^{\text{return,range}}$, distinguishing clean directional moves from choppy bars. The only candidate to improve full-mean Sharpe over baseline.
- **MLP_D3_return_vol5** (same embedding as `D3_return_vol5`, but switch `ConvDuelingDQN` to `MLPDuelingDQN`): once `log_price` is removed, the logsignature becomes a compact set of mostly stationary features, so convolution offers limited benefit. An MLP can directly reweight low-order and cross-term features, improving single-seed performance, but the gain is less stable in the 5-seed mean.
- **MLP_R5_volprof** (swap `log_return` for norm. volume): volume cross-terms encode order-flow dynamics [18], yielding the strongest Return (100.12%), but removing `log_return` eliminates oscillation-quality conditioning and no seed beats baseline on both metrics simultaneously. The MLP is especially suitable here because the R5 logsignature representation has already mixed information across channels and truncation levels, so much of the useful structure is built into the features. As a result, direct coordinate-wise reweighting is more effective than adding extra convolution.

Within this fold-level ablation, these results suggest that coordinate stationarity and cross-term semantics jointly shape what market structure the signature can express. They motivate, but do not by themselves validate, a future multi-asset design that prioritizes stationary coordinates and interpretable cross-terms rather than simply adding more raw market channels.

2.6 Limitations and Future Work

Several limitations should guide how these results are interpreted. First, the final trading experiments remain single-asset Bitcoin studies. This design was useful for isolating environment accounting, representation design, and agent behavior, but it does not test cross-asset allocation, correlation structure, liquidity differences, or portfolio-level risk budgeting. Second, all reported trading results are based on simulated execution assumptions. The environment includes fees, slippage, and exposure limits, but it does not fully model exchange outages, order-book depth, partial fills, market impact, funding rates, tax treatment, or operational latency.

The evaluation protocol also reduces, but cannot eliminate, overfitting risk. Walk-forward folds and multiple seeds make the results more credible than a single train/test split, yet the number of regimes is still limited by the available Bitcoin history. The representation experiment shows that signature features can preserve within-window timing information, but that finding is not equivalent to proving that every downstream trading policy will improve.

Future work should therefore extend the framework in three directions. The first is a multi-asset portfolio environment with explicit allocation constraints and portfolio-level drawdown controls. The second is a more realistic execution simulator that incorporates liquidity, funding, and stress scenarios. The third is broader validation across assets, folds, and market regimes, including paper-trading style evaluation before any live deployment is considered.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017. [Online]. Available: https://arxiv.org/abs/1707.06347.

[2] V. Mnih, K. Kavukcuoglu, D. Silver, *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015, doi: 10.1038/nature14236.

[3] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 30, no. 1, pp. 2094–2100, 2016, doi: 10.1609/aaai.v30i1.10295.

[4] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. 33rd Int. Conf. Mach. Learn. (ICML)*, PMLR, vol. 48, pp. 1995–2003, 2016. [Online]. Available: https://proceedings.mlr.press/v48/wangf16.html.

[5] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016. [Online]. Available: https://arxiv.org/abs/1511.05952.

[6] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, PMLR, vol. 80, pp. 1861–1870, 2018. [Online]. Available: https://proceedings.mlr.press/v80/haarnoja18b.html.

[7] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[8] A. Millea, "Deep reinforcement learning for trading—a critical survey," *Data*, vol. 6, no. 11, Art. no. 119, 2021, doi: 10.3390/data6110119.

[9] N. Pippas, E. A. Ludvig, and C. Turkay, "The evolution of reinforcement learning in quantitative finance: A survey," *ACM Comput. Surv.*, vol. 57, no. 11, Art. no. 295, 2025, doi: 10.1145/3733714.

[10] K. Olorunnimbe and H. Viktor, "Deep learning in the stock market—a systematic survey of practice, backtesting, and applications," *Artif. Intell. Rev.*, vol. 56, pp. 2057–2109, 2023, doi: 10.1007/s10462-022-10226-0.

[11] A. McLeod and T. Lyons, "Signature methods in machine learning," *EMS Surv. Math. Sci.*, vol. 13, no. 1, pp. 1–131, 2026, doi: 10.4171/EMSS/95.

[12] I. Chevyrev and A. Kormilitzin, "A primer on the signature method in machine learning," in *Signature Methods in Finance*, C. Bayer, G. dos Reis, B. Horvath, and H. Oberhauser, Eds. Cham, Switzerland: Springer, 2026, doi: 10.1007/978-3-031-97239-3_1.

[13] J. Morrill, A. Fermanian, P. Kidger, and T. Lyons, "A generalised signature method for multivariate time series feature extraction," arXiv:2006.00873, 2020. [Online]. Available: https://arxiv.org/abs/2006.00873.

[14] T. Lyons, "Differential equations driven by rough signals," *Rev. Mat. Iberoam.*, vol. 14, no. 2, pp. 215–310, 1998.

[15] B. Hambly and T. Lyons, "Uniqueness for the signature of a path of bounded variation," *Ann. Math.*, vol. 171, no. 1, pp. 109–167, 2010.

[16] I. Perez Arribas, "Derivatives pricing using signature payoffs," arXiv:1809.09466, 2018. [Online]. Available: https://arxiv.org/abs/1809.09466.

[17] M. Parkinson, "The extreme value method for estimating the variance of the rate of return," *J. Bus.*, vol. 53, no. 1, pp. 61–65, 1980.

[18] L. G. Gyurkó, T. Lyons, M. Kontkowski, and J. Field, "Extracting information from the signature of a financial data stream," arXiv:1307.7244, 2013. [Online]. Available: https://arxiv.org/abs/1307.7244.

[19] H. Gu, X. Guo, T. L. Jacobs, P. Kaminsky, and X. Li, "Transportation marketplace rate forecast using signature transform," *INFORMS J. Appl. Anal.*, vol. 55, no. 5, pp. 424–436, 2025, doi: 10.1287/inte.2025.0251.